

Exercícios

Exercício 1 - Comparação entre Bag-of-Words, TF-IDF e Word Embeddings

Considere os dois textos a seguir, após pré-processamento (remoção de stopwords, normalização

e tokenização): Texto A: ["produto", "bom", "ser", "produto", "qualidade"]

Texto B: ["produto", "ruim", "defeito"]

Considere o vocabulário:

["produto", "bom", "ruim", "ser", "qualidade", "defeito"]

Responda às questões a seguir.

Questão 1 – Bag-of-Words

- a) Represente os dois textos usando o modelo Bag-of-Words.
- b) Com base apenas nesses vetores, é possível afirmar que os textos possuem significados semelhantes ou diferentes? Justifique.

Resposta:

- a) Representação Bag-of-Words: Texto A [2, 1, 0, 1, 1, 0] e Texto B [1, 0, 1, 0, 0, 1]
- b) Não é possível afirmar que os textos possuem significados semelhantes ou diferentes, tendo em vista que o modelo não captura o significado ou relações semânticas das palavras, apenas a frequência das palavras no vocabulário.

Questão 2 – Frequência de termos

Represente o texto usando frequência de termos.

Resposta:

Vetor de frequência: Texto A [2, 1, 0, 1, 1, 0] e Texto B [1, 0, 1, 0, 0, 1]

Questão 3 – TF-IDF (conceitual)

Suponha que, em um corpus maior, a palavra "produto" apareça em praticamente todos os documentos, enquanto "defeito" apareça em poucos.

- a) Qual termo tende a receber maior peso no TF-IDF?
- b) Justifique o motivo, sem usar fórmulas.

Resposta:

- a) O termo "defeito" tende a receber maior peso no TF-IDF.

b) O TF-IDF atribui maior importância a palavras que aparecem com frequência em um documento, mas que são raras no restante do corpus. Como a palavra “produto” aparece em praticamente todos os documentos, ela é considerada muito comum e recebe um peso menor. Já a palavra “defeito”, por aparecer em poucos documentos, é mais específica e informativa, recebendo um peso maior.

Questão 4 – Word Embeddings (conceitual)

Considere agora um modelo de Word Embeddings bem treinado.

a) O que se espera da distância vetorial entre:

- “bom” e “ótimo”
- “bom” e “ruim”

b) Por que esse tipo de relação não pode ser capturada adequadamente por

BoW ou TF-IDF?

Resposta:

a) “bom” e “ótimo” → espera-se pequena distância vetorial, pois possuem significados semelhantes.

“bom” e “ruim” → espera-se maior distância vetorial, pois possuem significados opostos.

b) Porque BoW e TF-IDF são modelos baseados em contagem de palavras, tratando cada termo como um símbolo independente. Assim, eles não capturam relações semânticas entre palavras. Já os Word Embeddings representam palavras em um espaço vetorial onde palavras semanticamente semelhantes ficam próximas, permitindo capturar relações de significado entre elas.

Exercício 2 – Do texto bruto ao vetor (exercício-ponte)

Objetivo: Evidenciar que o notebook da Aula 3 começa exatamente onde a Aula 2 termina.

Enunciado: Considere o seguinte texto bruto:

“O produto não é bom, mas a qualidade do material é excelente.”

a) Aplique as etapas de pré-processamento vistas na Aula 2:

- normalização (caixa baixa);
- remoção de pontuação;
- tokenização;
- remoção de stopwords.

b) Explique por que apenas esses tokens, e não o texto original, são utilizados como entrada no notebook de Word Embeddings da Aula 3.

Resposta:

a) Etapas de pré-processamento

Normalização (caixa baixa):

o produto não é bom mas a qualidade do material é excelente

Remoção de pontuação:

o produto nao e bom mas a qualidade do material e excelente

Tokenização:

```
['o', 'produto', 'nao', 'e', 'bom', 'mas', 'a', 'qualidade', 'do', 'material', 'e', 'excelente']
```

Remoção de stopwords (admitindo uma lista típica do português):

```
['o', 'produto', 'nao', 'e', 'bom', 'mas', 'a', 'qualidade', 'do', 'material', 'e', 'excelente']
```

b) Porque os modelos de Word Embeddings trabalham com tokens (palavras separadas) e não com o texto bruto. O pré-processamento remove ruídos como pontuação e padroniza as palavras, facilitando a análise e permitindo que cada palavra seja representada como um vetor numérico.

Exercício 3 – Tokens × Vocabulário do embedding (conexão direta com o notebook)

Use o notebook fastText fornecido para listar as três primeiras palavras associadas a cada token obtido no Exercício 2.

Resposta:

Adicione a seguinte célula no final no final do notebook:

```
tokens = ["produto", "não", "ser", "bom", "qualidade", "material", "excelente"]
for t in tokens:
    if has_word(t):
        print(f"\n{t} ->", most_similar(t, topn=3))
    else:
        print(f"\n{t} -> OOV (não carregado no subconjunto)")
```

Saída:

```
produto -> [('produto.', 0.612028956413269), ('produtos', 0.5632554292678833), ('cosmético', 0.5602588653564453)]
não -> [('nem', 0.7865183353424072), ('nunca', 0.7823182940483093), ('mas', 0.7465847730636597)]
ser -> [('sido', 0.7598056197166443), ('pode', 0.721897304058075), ('sendo', 0.7199995517730713)]
bom -> [('ótimo', 0.809251070022583), ('excelente', 0.7200700044631958), ('otimo', 0.7020905017852783)]
qualidade -> [('Qualidade', 0.6553677320480347), ('qualidade.', 0.6508893370628357), ('excelência', 0.6345738172531128)]
material -> [('Material', 0.651745080947876), ('materiais', 0.6465468406677246), ('didático', 0.6207355260848999)]
excelente -> [('ótimo', 0.8194359540939331), ('ótima', 0.8179750442504883), ('excecelente', 0.7942733764648438)]
```